LECTURE SLIDES

Computer organization

EduForge AI • Generated Teaching Material • September 15, 2026

SLIDE 1: Computer Organization

Subtitle: This lecture explains how the parts inside a computer work together to run programs.

Key themes: • How the CPU processes instructions • How data is stored and accessed in memory • How hardware design choices affect speed and capacity

SLIDE 2: LEARNING OBJECTIVES

By the end of this lecture, students will be able to:

- Describe the main components of a CPU and their roles.
- Explain how the ALU performs arithmetic, logical, and two's-complement operations.
- Identify why word alignment and structure padding matter for memory performance.
- Outline the purpose of virtual memory and how pages are mapped to physical RAM.

SLIDE 3: TOPIC INTRODUCTION & CONTEXT

- Modern computers are built from many tiny electronic switches that represent binary 0s and 1s.
- The CPU (Central Processing Unit) fetches, decodes, and executes the binary instructions that make programs run.
- Memory is organized into words, which are the natural data size the CPU can move in a single cycle.
- Alignment means placing each word at an address that is a multiple of its size, which speeds up access.
- Understanding these low-level details helps students see why some programs run faster than others.

SLIDE 4: CORE CONCEPT 1: CPU - The Brain of the Computer

- The CPU contains a Control Unit that decides which operation to perform next.
- Inside the CPU are General-Purpose Registers, tiny fast storage locations that hold data while it is being processed.
- The Control Unit sends data to the ALU (Arithmetic-Logic Unit) for calculation or logical testing.
- Each instruction is carried out in one or more clock cycles, a rhythmic "tick" that synchronizes the whole chip.
- The CPU can also communicate with coprocessors, such as the VFP unit, for specialized tasks like floating-point math.

SLIDE 5: CORE CONCEPT 2: ALU - Arithmetic-Logic Unit

- The ALU is a combinational circuit that produces a result in a single clock cycle for most operations.
- It can add, subtract, multiply, and perform logical functions such as AND, OR, XOR, and NOT on binary numbers.
- Subtraction is performed by adding the two's-complement (invert-bits-and-add-1) of the number to be subtracted.
- Bit-masking uses logical operations with a mask value to isolate, set, or clear particular bits in a word.
- The ALU receives two input operands (Arg1 and Arg2) and an operation selector that tells it which function to execute.

SLIDE 6: CORE CONCEPT 3: MEMORY ORGANIZATION - WORDS, ALIGNMENT, AND PADDING

- A memory word is the natural chunk of data the CPU handles; on many systems a word is 4 bytes (32 bits).
- When a word's address is a multiple of 4, the CPU can fetch it in a single read cycle; misaligned accesses may need two cycles.

- Structure padding inserts unused bytes so each field starts at its natural alignment boundary, creating an "unpacked" layout.
- A packed structure removes the padding, saving space but sometimes causing slower or illegal accesses on certain CPUs.
- In the example BUSINESS structure, the unpacked size is 120 bytes (with padding) while the packed size is 115 bytes (no padding).

SLIDE 7: DEEP DIVE & MECHANISMS - THE INSTRUCTION CYCLE

- The fetch stage reads the next instruction word from memory using the program counter (PC).
- The decode stage interprets the binary opcode and determines which registers and ALU operation are needed.
- The execute stage sends the required operands to the ALU, which produces a result in one clock cycle.
- The write-back stage stores the result into a destination register or memory location.
- Some instructions also involve the VFP coprocessor, which uses its own registers (S0-S31 for single-precision, D0-D15 for double-precision).

SLIDE 8: REAL-WORLD APPLICATIONS & EXAMPLES

- Game developers rely on the VFP coprocessor's 32 single-precision registers to perform fast 3-D graphics calculations.
- Embedded systems often use packed structures to conserve memory, but must ensure the processor supports unaligned accesses.
- Operating systems implement virtual memory so each program thinks it has a large, private address space, improving security and multitasking.
- Page tables map 4 KB virtual pages to physical RAM pages, allowing the OS to swap rarely used pages to disk without the program noticing.
- The BUSINESS structure example shows how alignment adds 5 bytes of padding, which can matter in memory-constrained devices.

SLIDE 9: CRITICAL THINKING & ANALYSIS

- Why might a programmer choose a packed structure even though it can cause slower memory accesses on some CPUs?
- How does two's-complement representation simplify the hardware design of the ALU compared to separate subtraction circuitry?
- In what situations could misaligned memory accesses lead to program crashes rather than just slower performance?
- Consider the trade-off between having many registers (e.g., VFP's 32 single-precision registers) and the silicon area they occupy on a chip.
- How does virtual memory enable a computer with 4 GB of RAM to run a program that appears to need 8 GB of address space?

SLIDE 10: SUMMARY & CONCLUSION

Key points covered:

- The CPU's Control Unit, registers, and ALU work together to execute instructions in clock-synchronized steps.
- Alignment, word size, and structure padding directly affect how quickly data can be fetched and how much memory a program uses.
- Virtual memory abstracts physical RAM, providing isolation, flexibility, and the illusion of larger memory.

Discussion question: *If you were designing a new smartphone processor, which memory-alignment rule would you prioritize and why?*

Further reading: Explore an introductory textbook chapter on "Computer Architecture" or a reputable online tutorial that explains binary arithmetic and memory hierarchy.